

10-消息队列

消息队列面试题

1. 基础

问：为什么使用消息队列？

答：1. 异步：提升响应速度（如下单后异步发短信） 2. 削峰：缓冲突发流量（秒杀场景） 3. 解耦：生产者和消费者独立演进 4. 最终一致性：分布式事务（事务消息）

问：使用消息队列有什么缺点？

答：

缺点	说明
可用性降低	多依赖 MQ 组件，MQ 故障导致链路中断，需集群保障
复杂度上升	丢消息、重复消费、顺序、积压等需额外设计
数据一致性	生产成功但消费失败 → 数据不一致，需分布式事务/对账
运维成本	集群部署、监控、Topic 治理

原则：无银弹，有明确解耦/异步/削峰需求再用 MQ。

问：消息队列选型？

答：

MQ	特点	场景
Kafka	超高吞吐、日志型	日志收集、大数据
RocketMQ	事务消息、延迟消息	电商、金融
RabbitMQ	功能丰富、路由灵活	企业应用、复杂路由
Pulsar	存算分离、多租户	云原生

2. 通用问题

问：如何保证消息不丢失？

答：三个环节都要保证：

环节	方案
生产者	同步发送 + 确认 (ack)；事务消息
Broker	持久化（刷盘策略）；集群副本同步
消费者	手动 ACK；消费成功后再确认

问：如何保证消息不重复消费？

答：MQ 层面很难做到恰好一次，业务层保证 幂等：1. 唯一消息 ID + 数据库去重表 2. Redis SETNX 去重 3. 业务状态机（如订单状态只能流转一次） 4. 乐观锁（version 字段）

问：如何保证消息顺序？

答：1. 同一业务 key 路由到同一分区/队列（如 Kafka partition key、RocketMQ MessageQueueSelector） 2. 单线程消费 3. 消费失败不跳过，阻塞后续消息

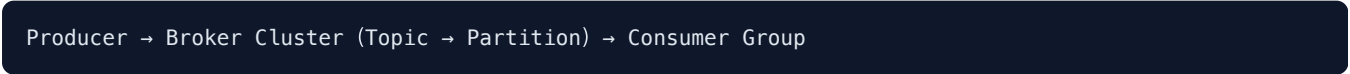
问：消息积压怎么处理？

答：1. 扩容消费者（不超过分区数） 2. 批量消费 3. 临时 Topic 扩容 + 转发 4. 降级非核心消费 5. 排查消费慢原因（DB 慢查询、死锁）

3. Kafka

问：Kafka 架构？

答：



- Topic：消息分类
- Partition：分区，有序；分区数决定最大并行度
- Consumer Group：组内消费者分摊分区，一条消息只被组内一个消费者消费

问：Kafka 为什么高吞吐？

答：1. 顺序写磁盘（Page Cache） 2. 零拷贝（sendfile） 3. 批量发送和压缩 4. 分区并行 5. 消费者拉取模式

问：什么是 ISR？

答：In-Sync Replicas，与 Leader 保持同步的副本集合。

- 生产者 `acks=all` 时，需 ISR 全部确认
- Leader 故障时从 ISR 中选举新 Leader（避免数据丢失）
- 落后太多的 Follower 会被踢出 ISR

问：Kafka 一个分区为什么只能被消费组内一个消费者消费？

答：保证 分区内消息顺序：若多个消费者同时消费同一 Partition，无法保证 offset 顺序提交，可能乱序。

关系：

消费者数 ≤ Partition 数（理想：相等，一对一）
消费者数 > Partition 数 → 部分消费者空闲
Partition 数 > 消费者数 → 一消费者可消费多 Partition

扩容：先增加 Partition，再增加 Consumer（Partition 数只能增不能减）。

问：Kafka 和 RocketMQ 消息确认机制有何不同？

答：

对比	Kafka	RocketMQ
生产者 ack	<code>acks=0/1/all</code>	同步/异步发送 + 刷盘/复制策略
消费确认	提交 offset（自动/手动）	CONSUME_SUCCESS / RECONSUME_LATER
重复消费	offset 提交后宕机可能重复	消费失败重试，默认 16 次进 DLQ
事务	幂等 Producer + 事务 API	事务消息（半消息+回查）

4. RocketMQ

问：RocketMQ 架构？

答：

```
graph TD
    Producer --> NS[NameServer (路由注册中心)]
    NS --> Broker[Broker (Master/Slave)]
    Broker --> Consumer
```

NameServer 无状态，Broker 主从部署。

问：RocketMQ 事务消息流程？

答：

1. 发送半消息 (Half Message) 到 Broker
2. 执行本地事务
3. 根据本地事务结果 Commit 或 Rollback
4. Broker 定期回查本地事务状态 (未收到 Commit/Rollback 时)

保证本地事务和消息发送的最终一致性。

问：RocketMQ 消息类型？

答：– 普通消息 – 顺序消息 – 延迟消息 (18 个延迟级别) – 事务消息 – 批量消息

5. RabbitMQ

问：RabbitMQ 核心概念？

答：

```
graph LR
    Producer --> Exchange
    Exchange --> Binding
    Binding --> Queue
    Queue --> Consumer
```

- **Exchange**：路由消息到队列
- **Queue**：存储消息
- **Binding**：Exchange 和 Queue 的绑定关系
- **Routing Key**：路由键

问：Exchange 类型？

答：

类型	路由规则
Direct	Routing Key 完全匹配
Fanout	广播到所有绑定队列
Topic	Routing Key 模式匹配 (* 一个词, # 多个词)
Headers	根据消息头属性匹配

问：RabbitMQ 如何保证可靠性？

答：1. 生产者 Confirm 模式 2. Exchange 和 Queue 持久化 3. 消息持久化（deliveryMode=2） 4. 消费者手动 ACK

问：RabbitMQ 死信队列和延迟队列原理？

答：死信（DLX）：消息满足以下条件进入死信队列： 1. 消费被拒绝且 `requeue=false` 2. 消息 TTL 过期 3. 队列达到最大长度

配置：声明 DLX Exchange，原队列设置 `x-dead-letter-exchange` 。

延迟队列（无原生支持，常用两种实现）：

方式	原理
TTL + DLX	消息设 TTL，过期后路由到死信队列消费
延迟插件	<code>rabbitmq_delayed_message_exchange</code>

6. 对比

问：Kafka、RocketMQ、RabbitMQ 对比？

答：

对比	Kafka	RocketMQ	RabbitMQ
吞吐量	最高	高	中
延迟	毫秒级	毫秒级	微秒级
事务消息	支持	原生支持	不支持
延迟消息	不支持	支持	插件支持
协议	自定义	自定义	AMQP
适用	日志、大数据	电商、金融	企业应用

7. 选型对比深入

问：消息队列选型需要考虑哪些维度？

答：

维度	考量
吞吐量	Kafka > RocketMQ > RabbitMQ
延迟	RabbitMQ 微秒级，Kafka/RocketMQ 毫秒级
可靠性	持久化、副本、ACK 机制
功能	事务消息、延迟消息、顺序消息、死信
运维	集群复杂度、监控、社区
协议	AMQP (RabbitMQ) vs 自定义 (Kafka/RocketMQ)
生态	Spring 集成、Cloud 原生

选型建议： – 日志/大数据/事件流 → **Kafka** – 电商/金融/事务消息 → **RocketMQ** – 复杂路由/企业集成 → **RabbitMQ** – 云原生多租户 → **Pulsar**

问：Kafka 和 RocketMQ 的消费模型差异？

答：

对比	Kafka	RocketMQ
消费模式	Pull（消费者主动拉）	Push（Broker 推，底层长轮询 Pull）
位移管理	__consumer_offsets 主题	Broker 端 / 消费者本地
重平衡	Consumer Group Rebalance	Rebalance（队列分配）
广播	不同 Group 各自消费	不同 Group 或 BROADCASTING 模式

8. Kafka 深入

问：Kafka 分区 Rebalance 是什么？有什么问题？

答：Consumer Group 内消费者变化（新增、下线、超时）时，**重新分配 Partition 给消费者**。

触发条件： – 消费者加入/离开 Group – 订阅 Topic 变化 – Partition 数量变化 – `session.timeout.ms` 超时未心跳

过程（旧版）：

1. 所有消费者停止消费 (Stop The World)
2. Coordinator 计算新分配方案
3. 通知各消费者新分配
4. 恢复消费

问题: Rebalance 期间不能消费, 大量消费者时停顿明显。

优化: – 增量 Rebalance (Kafka 2.4+ Cooperative Sticky): 只迁移变化的 Partition – 增大 `session.timeout.ms`、合理 `max.poll.interval.ms` – 静态成员 (`group.instance.id`) 减少不必要的 Rebalance

问: Kafka 如何保证分区内有序?

答: 1. 单 Partition 内有序: 消息按写入顺序追加到 Log 2. Producer 指定 key: 相同 key → 同一 Partition (`hash(key) % partition数`) 3. 单消费者线程消费该 Partition

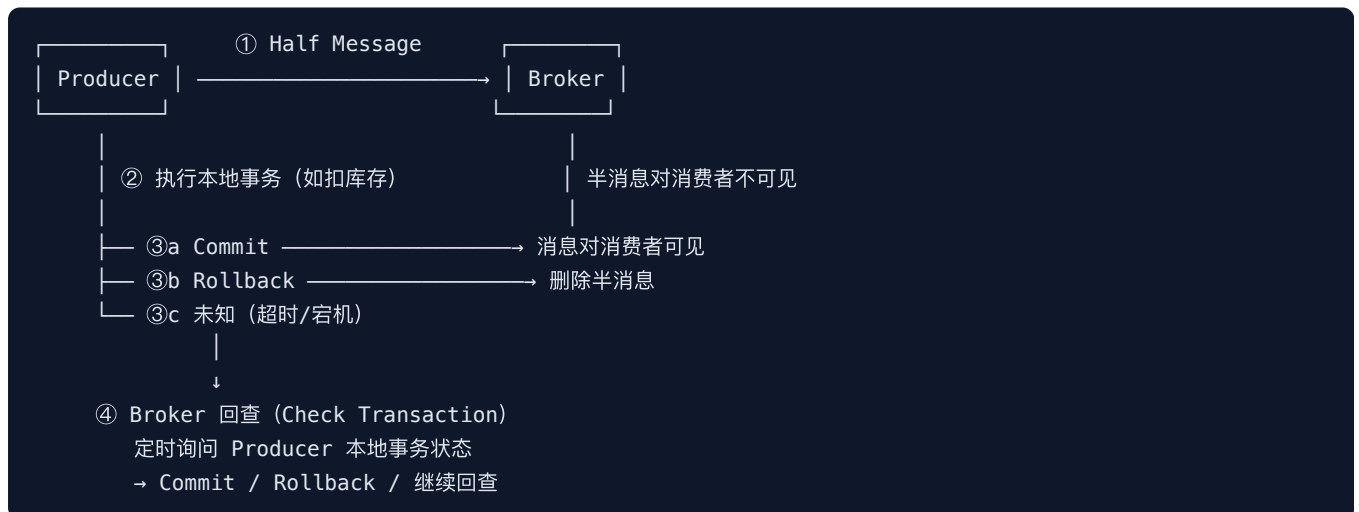
全局有序: Topic 只设 1 个 Partition, 牺牲并行度。

注意: Producer 重试可能导致乱序, 设置 `max.in.flight.requests.per.connection=1` 或使用幂等 Producer。

9. RocketMQ 深入

问: RocketMQ 事务消息详细流程?

答:



关键点: – 半消息写入 Op Topic (`RMQ_SYS_TRANS_HALF_TOPIC`), 对 Consumer 不可见 – 本地事务与半消息发送无原子性, 靠回查兜底 – 需实现 `TransactionListener`: `executeLocalTransaction` + `checkLocalTransaction` – 回查次数有限 (默认 15 次), 超限则丢弃

问: RocketMQ 延迟消息如何实现?

答: 18 个固定延迟级别: 1s 5s 10s 30s 1m 2m 3m 4m 5m 6m 7m 8m 9m 10m 20m 30m 1h 2h

原理：

- 1. 延迟消息先写入内部 Topic: SCHEDULE_TOPIC_XXXX
- 2. 按延迟级别分 18 个 Queue
- 3. ScheduleMessageService 定时扫描，到期后写入目标 Topic
- 4. 消费者正常消费

限制：不支持任意时间延迟，需自定义（如时间轮 + 数据库）。

Kafka：可用时间轮或外部调度（无原生延迟消息）。

问：RocketMQ 顺序消息如何保证？

答：全局顺序：Topic 只设 1 个 MessageQueue。

分区顺序：

- 1. Producer 发送时指定 MessageQueueSelector（如 `hash(orderId) % queueNum`）
- 2. 同一 `orderId` 的消息进同一 Queue
- 3. Consumer 使用 `MessageListenerOrderly`（加锁消费，失败挂起当前 Queue）

注意：消费失败会阻塞该 Queue 后续消息，需做好重试和死信处理。

10. 通用问题深入

问：什么是死信队列（DLQ）？如何处理？

答：消息多次消费失败后，进入死信队列（Dead Letter Queue），避免阻塞正常消费。

MQ	死信机制
RabbitMQ	<code>x-dead-letter-exchange</code> 绑定死信交换机
RocketMQ	<code>%DLQ%</code> 前缀 Topic，消费失败 16 次（默认）转入
Kafka	无原生 DLQ，需自建或配合 Kafka Connect

处理策略：1. 告警 + 人工排查 2. 修复后重新投递 3. 记录日志/落库，定期分析 4. 区分可重试异常 vs 不可重试（参数错误直接丢弃）

问：消息顺序性和性能如何权衡？

答：

策略	顺序性	性能
全局单 Queue/Partition	最强	最差
按业务 key 分区	分区内有序	中等
无顺序要求	无	最高

实践：订单状态变更按 `orderId` 分区有序；日志收集无需顺序，多 Partition 并行。

问：如何保证消息精确一次（Exactly Once）？

答：端到端精确一次极难，通常组合方案：

```
Kafka 幂等 Producer (PID + 序列号去重)
+ 事务 (原子写多 Partition)
+ 消费者手动提交 offset 与业务处理同一事务 (consume-transform-produce)
```

RocketMQ：业务层幂等 + 唯一消息 ID 去重。

实际：大多数场景 At Least Once + 业务幂等 即可。

问：消息积压时如何紧急扩容？

答：

```
1. 临时扩容 Consumer 实例 (≤ Partition/Queue 数)
2. 新建临时 Topic (更多 Partition)，写个转发程序将积压消息转过去
3. 新 Consumer Group 10~20 倍机器并行消费临时 Topic
4. 消费完后再切回正常流程
5. 同时排查慢消费根因 (DB、下游超时、GC)
```

注意：扩容 Consumer 超过 Partition 数无效，需先加 Partition。

问：消息中间件如何实现高可用？

答：

MQ	高可用方案
Kafka	多 Broker 集群 + Partition 多副本 + ISR；Controller 选举
RocketMQ	Master-Slave / Dledger (Raft 选主)；同步/异步复制
RabbitMQ	镜像队列 (Mirrored Queue)；Quorum Queue (Raft)

通用要点： 1. 多副本：消息写入多个节点再 ACK 2. **Leader 选举**：主节点故障自动切换 3. **生产者/消费者**：配置重试、多 Broker 地址 4. **监控告警**：Broker 宕机、ISR 收缩、消费 lag

图解加深

专题	要点
Kafka 架构	Broker、Topic、Partition、ISR、HW/LEO
消费者组	Rebalance 流程、Coordinator 选举
RocketMQ 事务	半消息、Op Topic、回查机制
延迟消息	SCHEDULE_TOPIC 时间轮扫描
消息中间件 HA	Kafka ISR、RocketMQ Dledger、RabbitMQ 镜像队列

大厂追问

1. **Kafka 为什么不适合做任务队列？** 日志型设计，删除策略按时间/大小而非消费确认；无原生延迟/优先级；Rebalance 影响消费。
2. **事务消息本地事务成功了，Commit 前宕机怎么办？** Broker 回查本地事务状态，根据结果 Commit 或 Rollback。
3. **消息发送成功但 DB 写入失败怎么处理？** 事务消息方案：先半消息 → 本地事务 → Commit/Rollback；或本地消息表 + 定时补偿。
4. **如何设计一个可靠的消息通知系统？** 至少一次投递 + 幂等消费 + 死信 + 监报告警 + 消息轨迹。
5. **Kafka 的 HW 和 LEO 是什么？** LEO 是 Partition 最后一条消息偏移；HW 是 ISR 中所有副本都已同步的最小 LEO，消费者只能读到 HW 之前。
6. **RocketMQ 刷盘策略？** 同步刷盘（可靠）vs 异步刷盘（性能）；同步复制 vs 异步复制。
7. **本地消息表方案是什么？** 业务表 + 消息表同一本地事务；定时任务扫描未发送消息投递 MQ；消费者幂等；最终一致，无 MQ 原生事务依赖。
8. **消息队列用了什么设计模式？** 观察者（发布订阅）、生产者-消费者、中介者（Broker 解耦）。
9. **Kafka 分区数如何设置？** 吞吐量 = Partition 数 × 单 Partition 吞吐；过多 Partition 增加文件句柄和 Rebalance 开销；一般按目标吞吐和 Consumer 数估算。